

Neural Architecture Search for Adaptive Website Fingerprinting

Gurjot Singh
University of Waterloo
Waterloo, ON, Canada
gurjot.singh1@uwaterloo.ca

Rob Jansen
U.S. Naval Research Laboratory
Washington, D.C., USA
robert.g.jansen7.civ@us.navy.mil

Diogo Barradas
University of Waterloo
Waterloo, ON, Canada
diogo.barradas@uwaterloo.ca

Abstract

Despite the use of encryption, the low-latency nature of the Tor anonymity network leaves users' traffic exposed to traffic analysis attacks aimed at uncovering the websites visited by users. These attacks, known as website fingerprinting (WF), have increasingly relied on deep neural networks (DNNs) due to their effectiveness in identifying complex traffic patterns. Thus far, however, the development of suitable DNN architectures for WF attacks has resorted to trial-and-error, potentially overlooking more effective and/or efficient attack models. In this paper, we introduce CRICKET, a framework for the automatic exploration of DNNs tailored to complex WF tasks. CRICKET leverages and adapts DARTS, a prominent neural architecture search (NAS) approach, to discover new DNN architectures that perform best for a given WF task. In evaluating CRICKET, we find that: (1) in a closed-world setting, CRICKET consistently outperforms the SOTA attacks by 2–7 points and up to 13 points against WF defenses; (2) in an open-world setting, CRICKET offers comparable performance to SOTA attacks against SOTA defenses; and (3) the novel DNNs produced by CRICKET use 1/3 to 1/2 as many parameters as the SOTA models. Our results show how CRICKET adapts to the WF context to produce novel DNNs that are both more effective and more compact than existing models.

CCS Concepts

• **Networks** → **Network privacy and anonymity**; • **Computing methodologies** → **Neural networks**.

Keywords

neural architecture search; Tor; website fingerprinting

ACM Reference Format:

Gurjot Singh, Rob Jansen, and Diogo Barradas. 2026. Neural Architecture Search for Adaptive Website Fingerprinting. In *25th Workshop on Privacy in the Electronic Society (WPES '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3847192.3847366>

1 Introduction

To safeguard their online privacy, users often rely on encrypted tunnels provided by low-latency anonymity networks like Tor [25]. These tunnels establish multi-hop encrypted pathways, masking both the content and destination of users' communications from

eavesdroppers. However, Tor's design, which prioritizes low-latency for a smoother browsing experience, retains key timing and volume characteristics of network packet exchanges. These website-specific traffic patterns remain visible through the encrypted tunnel [57], making Tor vulnerable to traffic analysis attacks known as website fingerprinting (WF) [54]. Through WF attacks, which recently employ deep neural network (DNN) classifiers [7, 54, 69, 72, 75, 80], eavesdroppers aim to match user traffic patterns with previously recorded website fingerprints to reveal the websites users visit.

So far, the search for DNN architectures applicable to WF has typically been guided by domain expertise coupled with trial-and-error choices [80], (e.g., adopting designs from unrelated domains such as computational vision [36] and biology [34]), rather than relying on a principled exploration of the broader architectural space. We argue that relying solely on heuristic choices or on existing architectures may lead researchers to overlook unconventional or innovative DNN designs, potentially missing out on breakthroughs that could significantly improve WF attack efficacy. Furthermore, increasingly sophisticated WF attacks still build on rather old DNN architectures. In particular, Deep Fingerprinting (DF), one of the first DNN architectures for WF, was published in 2018 [80] but is still used as the central component in several later attacks [2, 45, 69, 81] and as a major part of comparative evaluations and benchmarks even in the most recent studies [24, 41]. (See §2.2 for a thorough rundown on the role of DF in contemporary attacks.) The use of old model architectures that do not leverage modern advances leads us to question the relevance of WF evaluations that rely on them.

To address architectural limitations, we introduce CRICKET, a WF attack framework designed to dynamically discover novel DNN architectures and evaluate their real-world effectiveness. CRICKET employs DARTS [50], a neural architecture search (NAS) approach that *automates* the design of DNNs. Unlike manually crafted architectures that may overfit to synthetic datasets or specific feature representations, DARTS enables CRICKET to automatically explore a large design space and identify the architectures that perform best for a given dataset. As a result, CRICKET is highly *adaptive*: not only can it discover new DNN architectures considering the latest available datasets today, it can also be used to continuously discover the most relevant architectures as traffic features fluctuate over time. For example, when new datasets, features, or defenses are introduced, CRICKET can simply re-run its search phase to obtain new architectures tailored to the new environment.

CRICKET was designed specifically for the Tor WF domain. First, CRICKET encodes encrypted Tor network streams into one of many SOTA 1D sequential cell trace representations that capture the directional and temporal characteristics of each cell (i.e., packet) in the stream. Second, the DARTS search algorithm, which was originally designed for 2D image-like inputs, is adapted to consider 1D primitives across six architectural axes. By focusing CRICKET on 1D

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor, or affiliate of the United States government. As such, the United States government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for government purposes only. Request permissions from owner/author(s).

WPES '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-3026-9/2026/11

<https://doi.org/10.1145/3847192.3847366>

inputs, we ensure that it does not improperly learn 2D spatial correlations which Tor traces are not known to exhibit. Third, CRICKET conducts NAS over the 1D inputs to generate a DNN architecture that is fully adapted to the WF context in which it was trained.

We conduct an extensive evaluation of CRICKET across both synthetic [72] and genuine [43] datasets in which we compare CRICKET performance to four WF attacks [54, 69, 75, 80]. When using real-world, genuine data to find new DNNs, we find that CRICKET consistently outperforms the SOTA attacks by 2–7 percentage points (pp) in a close-world setting while offering performance comparable to the SOTA in a larger, open-world setting. The DNNs produced by CRICKET use fewer than 2M parameters on average, which is approximately 1/3 to 1/2 of the size of the SOTA models. These results hold across five WF defenses [9, 10, 31, 40, 47]: CRICKET outperforms Laserbeak [54] by up to 13 pp in the closed-world, and achieves comparable or slightly better performance in the open-world with smaller DNN models. The improved performance and model compactness demonstrate CRICKET’s ability to adapt: it can find DNNs that are smaller and/or better exploit the information available in the network traces, depending on the WF context.

Contributions. We summarize our contributions below.

- We design CRICKET, a NAS-based WF attack framework targeting realistic Tor network traces.
- We customize DARTS, a prominent NAS approach, to support 1D primitives compatible with Tor cell trace inputs.
- We evaluate CRICKET’s effectiveness, transferability, and robustness w.r.t. closed- and open-world settings, synthetic and genuine Tor traces, four WF attacks, and five WF defenses.

2 Background and Related Work

2.1 Website Fingerprinting

Website fingerprinting (WF) can be framed as a classification task where an eavesdropper aims to identify which webpage is visited by a user through some encrypted tunnel (see Figure 1). In the context of Tor, the adversary intercepts the communication between a user and the entry node of their Tor circuit. For mounting the attack, the adversary repeatedly visits a set of websites it wishes to monitor, collects the corresponding network traces, and extracts features to build a *fingerprint* database. Then, the adversary trains a model to predict which websites correspond to a given fingerprint. When a user visits a website via Tor, the adversary extracts the observed trace’s fingerprint and applies the model for classification.

WF research often adopts the *closed-world* assumption, where users visit only the pages the attacker has trained on, known as *monitored* pages. In the more realistic *open-world* scenario, users are assumed to access other *unmonitored* pages outside the attacker’s monitored set [46, 65]. Typically, a directional-time traffic trace representation is provided as input to a WF classifier, i.e., a time-ordered sequence of tuples (t_i, d_i) , where t_i encodes the time that the i th cell was observed and d_i encodes the direction it was forwarded. Recently, a multi-channel input format has been proposed and was shown to provide richer context about Tor cell traces [54]. **Website fingerprinting attacks.** Tor’s fixed-size cells (514 bytes) renders early WF attacks that require a packet-length distribution ineffective. Researchers then developed techniques to overcome

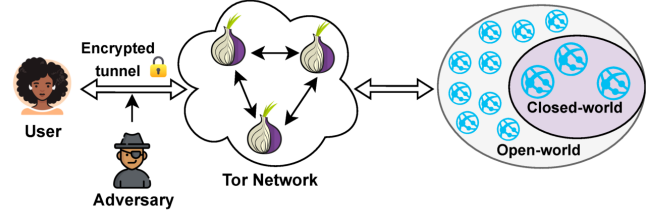


Figure 1: Website fingerprinting threat model.

Tor’s encryption, first by relying on manual feature engineering and later using deep learning to automatically extract features.

Attacks with manual feature extraction. Numerous techniques rely on manually designed features to train ML classifiers and fingerprint Tor website traces. These include the use of a variable n-gram Naïve Bayes classifier [27], support vector machines (SVMs) [11, 62, 63, 88], k-Nearest Neighbors (k-NN) [35, 87], and random forests [35]. These attacks considered a varying number of features manually derived from packet timing and directions, with up to 4,200 considered by Wang et al. [87]. It quickly became difficult to develop and understand the importance of individual features.

Attacks with automated feature extraction. More recently, WF research has shifted towards deep learning approaches which automatically extract features from different trace representations. The main techniques that are considered include stacked denoising autoencoders (SDAEs) [1, 72], recurrent neural networks (RNNs) [72], convolutional neural networks (CNNs) [7, 69, 72, 75, 80], and methods based on transformers [54]. These methods represent the SOTA WF attacks when applied to the latest Tor trace datasets.

Website fingerprinting defenses. WF defenses obfuscate the characteristics of website access traces, thereby helping preserve browsing privacy against traffic analysis. While we overview WF defenses, we refer the reader to the work of Mathews et al. [55] for a more comprehensive analysis of the WF defense landscape.

Padding-based defenses. These obscure timing patterns and packet bursts (consecutive same-direction packets) by enforcing consistent transmission rates. Defenses can pad at regular [10, 11, 40], semi-regular [9], randomized [31], and adaptive intervals [46, 77].

Subsequence-based defenses. Defenses that attempt to make traffic sequences within a collision set indistinguishable to the attacker by producing similar outputs for the *shortest common supersequence* of the set include Glove [59], Supersequence [87], and Walkie-Talkie [89]. These defenses require the need of prior knowledge about the network traces/burst sequences of the websites they are meant to protect, a challenging requirement in practice [69].

Application-based defenses. Defenses that obfuscate application-based properties such as request order and page size and that are agnostic of Tor include HTTPoS [51], LLaMA [14], ALPaCA [14], and a defense introduced by The Tor Project [64].

Splitting-based defenses. These limit an adversary’s observations by splitting a traffic stream, e.g., across two independent Internet access points [37], or across multiple entry relays or circuits [19].

Trace generation-based defenses. Defenses that use generative models have been proposed to foil WF attacks that rely on deep learning, e.g., by generating perturbed but realistic network traces [32, 58] or adversarial traces designed to subvert attack models [39, 68, 74].

2.2 The Role of DF in Later WF Attacks

Table 1 surveys WF attacks published since Deep Fingerprinting (DF) [80], identifying the role DF plays in each in order to characterize how often it serves as a building block or as part of comparative evaluations. The table shows a heavy reliance on DF’s design; DF is the working model behind many later attacks, and one of the main baselines against which nearly all of the rest measure progress.

Sourcing of related publications. We center on papers that describe WF attacks on Tor, published in the “big-four” security venues (IEEE S&P, ACM CCS, USENIX Security, NDSS), as well as the PoPETs and IEEE TIFS journals. We exclude papers focusing on pure WF defense development. We drew candidates from a community bibliography accompanying a recent WF survey [18] and from the Website-Fingerprinting-Library repository [20].

Categories. We categorize each paper to reflect the role DF plays in its contribution. We use five categories, ordered in decreasing order by how central DF is to the paper’s contribution: *original DF* (–), the original DF paper; *DF as backbone* (■), where the DF network or its convolutional trunk is the attack’s classifier or feature extractor; *one of several base attacks / enhanced* (▲), where DF is run as a component but interchangeably with other classifiers or as a base for a wrapper or adaptation scheme; *comparison baseline* (○), where the proposed attack is a distinct model and DF is re-run only for comparison, and; *object of study* (◇), where DF is run as a target of measurement or analysis rather than as the authors’ attack.

Model architecture stagnation. The table exposes how little the underlying architecture has changed. DF is the classifier or feature extractor in attacks for few-shot matching [81], timing-based inputs [69], and self-supervised pretraining [2]; even attacks built around a Transformer keep DF’s convolutional trunk [45, 101]. A second group builds on top of DF, wrapping or fine-tuning it for new settings [23, 56, 67, 82]. The attacks that introduce a different model still measure progress against DF, which serves as baseline in recent works [21, 22, 24, 54, 75, 76]. The designs that depart from it, such as Transformers [22, 54], state-space models [24], and contrastive encoders [21], remain isolated cases, and each is validated in large part by improving over DF and a small set of its successors. The field has advanced traffic representations, training regimes, and threat models, but has revisited the underlying architecture far less often. This gap motivates our search over the architecture space rather than the continued reliance on a single DNN design.

2.3 Primer on Neural Architecture Search

Automated Machine Learning (AutoML) simplifies the conventional machine learning workflow by using different optimization techniques to automate feature engineering, model selection, hyperparameter tuning, and model evaluation [92]. Within the realm of AutoML, neural architecture search (NAS) addresses the task of automatically designing neural network architectures tailored to specific tasks. After experiencing recent success in areas such as computer vision or natural language processing, NAS expanded into multiple domains, including network traffic analysis [52, 53, 100]. **NAS vs. hyperparameter optimization.** NAS is closely related to hyperparameter optimization [6]. However, while the latter focus on optimizing scalar hyperparameters, NAS targets architectural choices [92]. In practice, this means searching over neural

Table 1: WF attacks at top-tier security and privacy venues since Deep Fingerprinting (DF) was introduced in 2018.

Year	Venue	Work	Role of DF
2018	CCS	[80]	– Original DF paper
2018	CCS	[49]	◇ Study (corroborates information-leakage analysis)
2019	CCS	[81]	■ DF backbone (embeds traces for N -shot matching)
2019	PETS	[7]	○ Baseline (work introduces a ResNet-based CNN)
2019	PETS	[61]	○ Baseline (work introduces a CNN/DNN architecture)
2020	S&P	[85]	○ Baseline (DF compared to precision-optimized attacks)
2020	PETS	[69]	■ DF backbone (classifies new timing features)
2020	PETS	[67]	▲ DF as base (augmented by a website oracle)
2021	CCS	[86]	◇ Study (defense evaluation)
2021	PETS	[60]	○ Baseline (introduces a semi-supervised GAN)
2021	PETS	[82]	▲ DF as base (one of several, QUIC setting)
2022	USENIX	[15]	■ DF backbone (via TF, adapted to the online setting)
2022	TIFS	[90]	■ DF backbone (ensemble of DF snapshots)
2023	CCS	[2]	■ DF backbone (self-supervised pretraining)
2023	CCS	[45]	■ DF backbone (CNN stem feeding a Transformer)
2023	USENIX	[75]	○ Baseline (introduces the TAM representation)
2023	S&P	[22]	○ Baseline (introduces the Trans-WF transformer)
2023	S&P	[55]	◇ Study (defense SoK)
2023	PETS	[42]	◇ Study (dataset built via simulation)
2024	CCS	[21]	○ Baseline (introduces a contrastive encoder)
2024	CCS	[101]	■ DF backbone (metric-learning feature transform)
2024	USENIX	[56]	▲ DF as base (boosted with subpage sets)
2024	TIFS	[54]	○ Baseline (introduces a multi-channel transformer)
2025	CCS	[76]	○ Baseline (introduces the CIF representation)
2025	S&P	[24]	○ Baseline (introduces a state-space model)
2026	NDSS	[41]	■ DF backbone (classifies transduced traces)
2026	NDSS	[23]	▲ DF as base (fine-tuned against drift)

Role of DF: – original DF, ■ DF is the attack’s backbone, ▲ one of several base attacks / enhanced, ○ comparison baseline, ◇ object of study.

network topologies, often represented as directed acyclic graphs where nodes and edges are annotated with primitive operations such as convolutions, pooling layers, or skip connections.

NAS pillars. NAS approaches are built on top of three key pillars: search space, search strategy, and performance estimation [29, 93].

Search space refers to the collection of operations (e.g., convolution, pooling) and their parameterizations (e.g., kernel sizes, strides) that a NAS algorithm can explore towards converging into a suitable neural network architecture for solving the task at hand. A smaller search space defined by domain knowledge experts facilitates NAS algorithms in quickly converging to high-performing architectures. Conversely, larger search spaces with primitive building blocks may require a longer runtime, but hold the potential for uncovering novel and more effective architectures [71]. Today, *cell-based* search spaces are a predominant type of search space in NAS, taking inspiration from the repetitive patterns found in SOTA human-designed CNNs. Rather than searching for an entire network architecture from scratch, this approach involves searching over small cells, which are then stacked to form the final architecture [103].

Search strategy outlines how the exploration of the search space is conducted. In essence, it is an optimization process aimed at identifying high-performing architectures within the defined space. Two primary categories of search strategies exist: *black-box* optimization-based techniques, and *one-shot* techniques. While the former are based on the independent training of numerous architectures (e.g., via random search [5], genetic algorithms [70], bayesian optimization [96], or Monte Carlo tree search [84]), the latter use the concept of *supernets* that include all potential architectures within a search space as subnetworks, effectively enabling the evaluation of multiple architectures without the need to train each one from scratch. Supernet’s training costs rise linearly, despite the number of subnets in the supernet growing exponentially [92].

Performance estimation schemes encompass methods that swiftly predict the performance of the architectures under exploration,

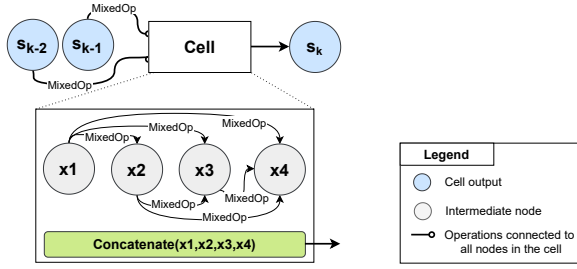


Figure 2: Overview of a normal DARTS cell, showing four intermediate nodes with edges carrying candidate primitive operations and output given by their concatenation.

avoiding the need to fully train each of these architectures. Proxy metrics such as shorter training, reduced data, or lower input resolution [16, 48, 98, 103] decrease costs but introduce bias, which may be acceptable if the relative performance between architectures remains stable. Learning-curve extrapolation accelerates evaluation by predicting final accuracy from partial training traces, e.g., with surrogate models [3, 48]. Network morphisms transfer weights between architectures [8, 28, 91], while *one-shot* NAS strategies train supernetworks from which subnets inherit their weights [4, 66, 73].

Placing it all together, NAS can be succinctly defined as the task of identifying an architecture a within search space $\mathcal{A}$, given a dataset $\mathcal{D}$, training pipeline $\mathcal{P}$, and a specified time or computation budget t [92]. The objective is to maximize validation accuracy under these constraints. Next, we describe a popular NAS method which we use as the main building block for our novel WF approach. **Differentiable ARchiTecture Search (DARTS)**. A popular one-shot NAS approach using supernetworks is Liu et al.’s DARTS [50]. It uses a cell-based search space, representing an architecture as a stack of n cells (with n being a configurable parameter), where each cell is a small directed acyclic graph (DAG). As per Figure 2, each cell takes as input the outputs of the two preceding cells and produces an output by combining multiple intermediate computations. Formally, a cell is a DAG $G = (V, E)$ with two input nodes (s_{k-2}, s_{k-1}) and N intermediate nodes $\{x^{(1)}, \dots, x^{(N)}\}$. The cell output (s_k) is the concatenation of all intermediate nodes, and each edge $(u \rightarrow v) \in E$ carries a *mixed operation*, i.e., a combination of candidate primitives such as convolutions, pooling layers, or skip connections:

$$\bar{o}_{u \rightarrow v}(x) = \sum_{o \in O} \pi_{u \rightarrow v}(o) o(x),$$

where the mixture weights are given by a softmax over the architecture parameters α :

$$\pi_{u \rightarrow v}(o) = \frac{\exp(\alpha_{u \rightarrow v}^{(o)})}{\sum_{o' \in O} \exp(\alpha_{u \rightarrow v}^{(o')})}.$$

During the model search phase, the network weights and architecture parameters (α) are optimized jointly via gradient descent; the weights are trained on the training set, while the architecture parameters are progressively updated on the validation set.

Since one-shot search is computationally intensive, it is common to downscale the network during exploration by reducing the number of filters per cell (width) and the number of stacked cells (depth), allowing efficient search without significantly compromising final performance [103]. To this end, DARTS organizes its cells into two

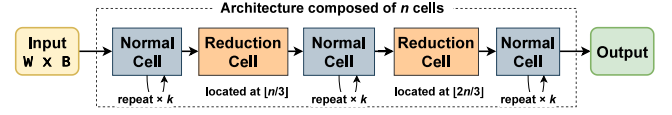


Figure 3: Final architecture stacking multiple cells together.

types: *normal cells* (as represented in Figure 2), which preserve spatial dimensions of the input feature maps, and *reduction cells*, which downsample feature maps (stride ≥ 2) to aggregate context.

In an architecture composed of n cells (see Figure 3), normal cells are repeated k times (depending on the desired total number of cells n) to preserve the temporal resolution of the input, while reduction cells halve the temporal dimension at predetermined locations ($\lfloor n/3 \rfloor$ and $\lfloor 2n/3 \rfloor$). Once the search concludes, the continuous mixtures on each edge are discretized by retaining the top- m incoming edges per node ($m = 2$) and selecting the single operation with the largest weight. The resulting (compact) normal and reduction cells are then stacked to form the final network, scaling up width and depth to increase capacity and enable richer feature extraction.

2.4 NAS for Website Fingerprinting

Due to their ability to find effective ML models, AutoML pipelines have been previously applied to the traffic analysis domain [26, 38] and specifically to WF [33], revealing promising results. However, the performance of “classical” ML classifiers has long been found to be subpar w.r.t. deep learning in the context of WF [72, 80].

Earlier NAS for WF. Singh et al. [79] explored the application of NAS to WF tasks. Their preliminary study relied on AutoKeras [44], an open-source NAS system that uses Bayesian optimization [30] to guide the discovery of suitable DNNs. Though insightful, Singh et al.’s early incursion into NAS for WF presents several limitations: *Rigid operational space.* While user-friendly, AutoKeras offers limited customization and flexibility, especially in defining the target NAS search space and strategy. Users face restrictions in adding new architectural layers, operations, or connections, limiting the exploration of new DNN architectures. This stems from AutoKeras’s reliance on predefined search spaces tailored to standard tasks such as image and text classification, making it difficult to handle non-traditional inputs. Notably, Singh et al. aggregated Tor cell traces into image-like 2D representations for ensuring compatibility with AutoKeras. In §3.3, we show that this choice is suboptimal and introduces challenges for accurate trace classification.

Severe computational overhead. AutoKeras adopts a multi-trial NAS strategy, where each trial corresponds to the independent training and evaluation of a different DNN architecture. This approach incurs substantial computational costs, as it requires fully training multiple candidate models before selecting the best one amongst all trials. In contrast, gradient-based methods such as DARTS (see §2.3) optimize architecture parameters in a continuous space, allowing efficient weight sharing across candidate DNN architectures, thus helping to drastically reduce the overall model search time.

Lackluster performance. Singh et al. showed that DNNs found by AutoKeras performed worse than Tik-Tok [69] on undefended Tor traffic, even when the discovered models were $\sim 6\times$ larger than the DF [80] model used by the Tik-Tok attack. When tuning for efficiency, DNNs that were smaller (and performed faster inference) than Tik-Tok suffered severe classification performance penalties.

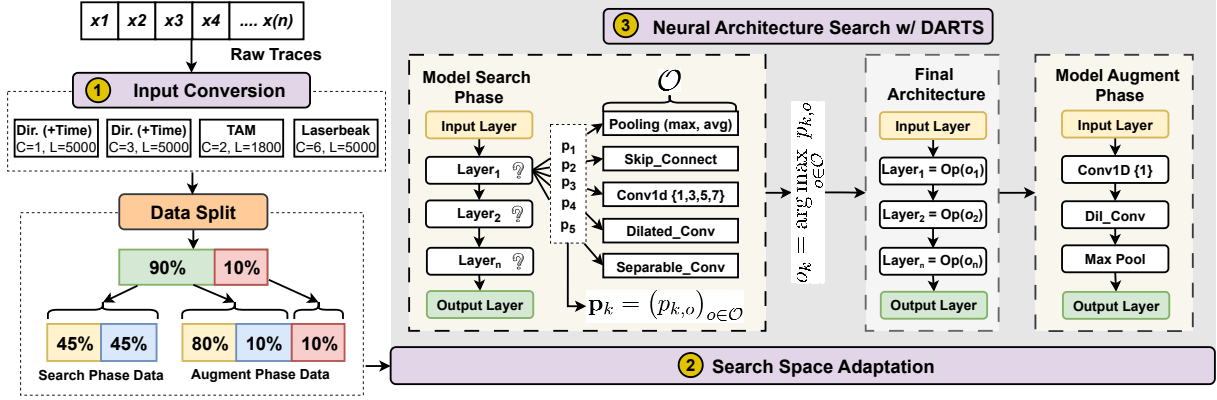


Figure 4: Overview of CRICKET’s execution pipeline. ① **Input Conversion:** Tor cell traces are transformed into 1D input representations that capture the traces’ characteristics using C feature channels and temporal length L . This data is then split into search and augment subsets. ② **Search Space Adaptation:** CRICKET redefines DARTS’ 2D operations into 1D, accommodating the transformed inputs. ③ **Neural Architecture Search:** First (search), CRICKET performs a bi-level optimization to jointly learn optimal weights and architectural parameters for a neural network model, ultimately deriving a concrete, final architecture. Second (augment), the final architecture is retrained, validated, and tested on the complete dataset.

3 Throwing DARTS at Website Traces

We now introduce CRICKET,¹ a new WF attack pipeline that uses NAS (specifically, DARTS) to identify anonymized website traces.

3.1 Overview of CRICKET

CRICKET harnesses NAS for tackling the WF task by adapting DARTS to operate directly on Tor cell traces. Since the effectiveness of WF attacks often depend on how traces are represented, CRICKET allows for the exploration of multiple input representations during the NAS search process rather than committing to a single one.

Our attack pipeline, represented in Figure 4, unfolds in three main stages: a) transforming raw traces into multiple input representations; b) adapting the DARTS search space to support the input representations; and c) running the architecture search to discover effective classification models. Next, we briefly describe each stage of our pipeline before expanding upon each of them.

Stage 1: Input conversion. We ingest raw Tor cell traces and convert them into different input representations that capture traffic’s directional and temporal characteristics. We do so for evaluating the cross-compatibility of our approach and determining which input representations yield the most effective models via NAS. In addition to SOTA trace representations proven effective in prior work (i.e., traffic aggregation matrices [75] and multi-channel representations [54]), we introduce a three-channel encoding that builds on the directional+timing input format [69] for explicitly tracking outgoing cells, incoming cells, and padding events.

Stage 2: Search space adaptation. Standard DARTS is designed for 2D inputs such as images. We adapt its search space dimensions so that it can naturally accommodate for 1D sequential data which is typical of Tor cell traces. Beyond adding new operations to the search space, we redefine the set of existing candidate operations (e.g., one-dimensional convolutions, pooling, and skip connections).

Stage 3: Neural architecture search. After defining DARTS’ search space, the last stage of our pipeline runs DARTS to jointly optimize network weights and architecture parameters. The search process aims to identify a set of normal and reduction cells that, when stacked, should form a final neural network architecture that is highly effective for solving WF tasks.

3.2 Converting Input Traces to Representations

While websites’ Tor cell traces are inherently *one-dimensional* (represented as a time-ordered sequence of cells with associated direction), past WF work has shown that these can be extended into more robust representations for fueling more accurate trace classification. Next, we describe four different trace representations compatible with CRICKET, including one we designed ourselves.

1-channel directional representation (baseline). Consists of the directional trace representation used by DF [80], where each Tor cell included in a trace is represented in sequence by a scalar:

$$x_t = \begin{cases} +1 & \text{if cell at } t \text{ is outgoing,} \\ -1 & \text{if cell at } t \text{ is incoming,} \\ 0 & \text{if padded.} \end{cases}$$

This representation can easily be extended into the direction+timing one used by Tik-Tok [69], by multiplying each scalar by inter-cell timings. This representation results in a 1D vector $X \in \mathbb{R}^L$, where L is a fixed trace length (e.g., 5,000). This encoding preserves directionality and ordering but conflates magnitude with the absence of traffic (due to padding), potentially limiting expressiveness.

3-channel directional representation (ours). To better disentangle the directional information provided by actual Tor cells from padding, we expand each scalar cell into a three-channel vector:

$$x_t = \begin{cases} [1, 0, 0] & \text{if cell at } t \text{ is outgoing,} \\ [0, 1, 0] & \text{if cell at } t \text{ is incoming,} \\ [0, 0, 1] & \text{if padded.} \end{cases}$$

¹Cricket is a popular darts game where players try to be the first to “close” specific numbers (usually 15–20 and the bullseye), then score points by hitting those numbers.

Here, the first channel activates only for outgoing cells, the second only for incoming, and the third explicitly marks padding data. Stacking over time yields $X \in \mathbb{R}^{3 \times L}$. This separation avoids sign ambiguity, enables 1×1 convolutions to fuse directions selectively, and allows CNNs to learn padding-invariant filters. Like our baseline, this representation can also be extended for encompassing direction+timing data. Empirically, we found this encoding to improve WF efficacy vs. the 1D channel representation (see Table 3–§5.1).

Traffic Aggregation Matrix (TAM). As our third input representation, we consider the Traffic Aggregation Matrix (TAM) structure of Shen et al. [75]. In this representation, time is divided into fixed-width bins of duration Δt . For each bin, we record the number of cells observed during that interval. This produces a sequence:

$$X = [x_1, x_2, \dots, x_B],$$

where each x_b represents the number of cells within the time range $[(b-1)\Delta t, b\Delta t)$. The resulting sequence captures burstiness and coarse timing while smoothing out jitter. TAM vectors reduce sensitivity to noise and to defenses that manipulate packet timings.

Laserbeak’s 6-channel representation. We also evaluate the six-channel input representation introduced in Laserbeak [54]. Each channel includes cell timing, cumulative sums, inter-arrival distributions, inverse-log inter-arrivals, directional encodings, and estimated running rates. For each cell p_t at position $t \in \{1, \dots, L\}$ with timestamp T_t , size S_t , and direction $d_t \in \{-1, +1\}$, we have:

$$x_t = [f_t^{(1)}, f_t^{(2)}, f_t^{(3)}, f_t^{(4)}, f_t^{(5)}, f_t^{(6)}]^T \in \mathbb{R}^6,$$

Stacking over the trace yields the tensor

$$X = [x_1, x_2, \dots, x_L] \in \mathbb{R}^{6 \times L},$$

which integrates traffic semantics. This representation improves WF attack efficacy by combining cell-level and statistical characteristics.

3.3 Adapting DARTS’ Search Space Dimension

DARTS’ capabilities have been aimed at image classification benchmarks where datasets are often comprised of squared images, e.g., CIFAR-10 (32×32) or MNIST (28×28). Accordingly, its search space and candidate operations are tailored to *two-dimensional* data, where convolutional operators naturally exploit spatial locality.

In contrast to images, WF datasets are typically comprised of *one-dimensional* Tor cell sequences. In this kind of input, characteristics such as ordering and directionality have been shown as highly informative, but no spatial correlation is known to exist. This difference in modality suggests that 2D convolutional kernels, which are effective in vision tasks, may not be well suited to website traces. Thus, we argue that deep learning architectures tailored for WF must explicitly capture sequential dependencies, burst patterns, and direction-based characteristics in a one-dimensional format.

Why not just convert website traces to 2D inputs? Motivated by the typical image workloads handled by DARTS, a simple strawman approach would be to convert 1D WF traces (e.g., of length 5,000) into a 70×70 image representation, thereby satisfying DARTS’ assumption of 2D convolutions on square grids. However, one may wonder whether the outcome of such a conversion provides spatial correlations that can be adequately leveraged by DARTS.

To reason whether such a reshape induces meaningful spatial structure, we compared the value of Moran’s I , a common metric

Table 2: Moran’s I for WF traces reshaped into 2D grids compared with image datasets, where $I \in [-1, 1]$. Greater values indicate stronger spatial autocorrelation.

Dataset	Moran’s I Mean	95% CI
AWF ₁₀₀ (70×70 , row)	0.281	(−0.016, 0.556)
AWF ₁₀₀ (70×70 , col)	0.281	(−0.016, 0.556)
GTT ₁₀₀ (70×70 , row)	0.366	(−0.021, 0.788)
GTT ₁₀₀ (70×70 , col)	0.366	(−0.021, 0.788)
CIFAR-10 (32×32)	0.836	(0.662, 0.945)
MNIST (28×28)	0.766	(0.610, 0.873)

for measuring spatial autocorrelation [13, 99] between two WF datasets’ traces (AWF₁₀₀ and GTT₁₀₀, each containing 100 traces of 100 different websites, being generated as subsets of AWF [72] and GTT [43], respectively—see §4.3) and common image datasets. Intuitively, this metric assesses whether neighboring values on a 2D grid exhibit systematic similarity, thus serving as a statistical test for gauging whether a dataset possesses true spatial relationships.

The results of our calculations, reported in Table 2, suggest that the reshaped traces do not exhibit strong spatial dependence: Moran’s I mean values for AWF₁₀₀ (≈ 0.28) and GTT₁₀₀ (≈ 0.37) are much less than those of CIFAR-10 (≈ 0.84) and MNIST (≈ 0.77). Moreover, row- and column-wise reshapes of the traces yield identical values, suggesting that any apparent correlation is an artifact of reshaping rather than an intrinsic property of the data.

Adapting the DARTS search space to 1D. To address potential pitfalls arising from a blind application of DARTS to reshaped 2D versions of network traces, we adapted its search procedure to natively operate on one-dimensional (1D) inputs. This enables DARTS to search for architectures tailored to the sequential and temporal structure of Tor cell traces. To this end, we redesigned DARTS’ cell-based search space along six axes (the candidate operation set $\mathcal{O}$ described below) to include 1D primitives such as separable convolutions, dilated convolutions for capturing longer-range dependencies, and pooling operations. Each primitive op is wrapped as $\text{ReLU} \rightarrow op \rightarrow \text{BatchNorm1d}$. Widening the search space to recurrent or attention-based operators is left to future work.

Standard 1D convolutions. We expanded the repertoire of standard 1D convolutions with kernel sizes $k \in \{1, 3, 5, 7, 9\}$ to capture temporal correlations at different time scales. Small kernels (e.g., $k = 1$) act as pointwise filters that perform cross-channel mixing at each time step without temporal context, while larger kernels capture Tor cell burst patterns and short inter-arrival dependencies.

Dilated depthwise-separable convolutions. We incorporate dilated convolutions for capturing longer request-response intervals that help efficiently model delayed cells and bursty traffic. A dilation factor d introduces gaps between consecutive kernel elements, such that the effective receptive field R becomes $R = d(k-1) + 1$.

Separable convolutions. These split a convolution into two steps: a) depthwise convolution, which filters each channel independently to capture fine-grained local patterns, and b) pointwise convolution, which combines channels to enable cross-channel feature integration. This decomposition strengthens local pattern extraction while increasing computational efficiency w.r.t. full convolutions.

Strided convolutions. These perform downsampling by skipping every other step (stride $s = 2$), reducing sequence length while still

learning which features to keep. Unlike fixed pooling, strided convolutions can adaptively determine which local features to retain while halving the temporal resolution, making them robust to jitter in trace alignments. The temporal output length L' is computed as

$$L' = \lfloor ((L + 2p - d(k-1) - 1)/s) + 1 \rfloor$$

where L is the input length, p is the amount of zero-padding, k is the kernel size, d is the dilation factor, and s is the stride.

Pooling. To attenuate sensitivity to small fluctuations in Tor cells' timing, we use max- and avg-pooling operators with kernel size $k = 3$, i.e., the width of the temporal window over which local statistics are aggregated; max-pooling selects the most salient activation in the window and avg-pooling computes a smoothed local mean.

Skip / None. The *skip* operator passes information directly across layers, ensuring signals are not lost as the network deepens. The *none* operator, in turn, removes a connection entirely, allowing the search process to prune unnecessary pathways.

CRICKET's 1D input representation. Ultimately, we represent each input trace as a tensor $X \in \mathbb{R}^{C_{in} \times L}$ with C_{in} channels (e.g., outgoing vs. incoming cells, or engineered feature channels as in Laserbeak) and temporal length L .

3.4 Neural Architecture Search

The final stage of CRICKET's pipeline executes two intrinsic phases of DARTS: a) model search, which identifies high-performing architectures by jointly optimizing network weights and architecture parameters; and b) model augmentation, which discretizes the learned continuous architecture into a cell structure that can be stacked, instantiated as a model, and trained from scratch.

Model search phase. The goal is to identify architectures that effectively model patterns in Tor cell traces, enabling accurate WF attacks. The search simultaneously optimizes network weights w , which parameterize the convolutional and transformation layers, and architecture parameters α , which determine which operations dominate each edge in the cell. Following Liu et al.'s [50], we allocate 90% of the dataset to the model-search phase and divide this subset equally into two disjoint partitions, corresponding to 45% of the full dataset each. The first partition is used to optimize the network weights w , while the second is used to optimize the architecture parameters α . The remaining 10% is held out from model search.

Bi-level optimization. The search is cast as an optimization problem:

$$\min_{\alpha} \mathcal{L}_{val}(w^*(\alpha), \alpha) \quad \text{s.t.} \quad w^*(\alpha) = \arg \min_w \mathcal{L}_{train}(w, \alpha),$$

where $\mathcal{L}_{train}$ and $\mathcal{L}_{val}$ denote losses over the training and validation sets of Tor traces, respectively. In this setup, weights w learn to fit the training data for a fixed architecture α , while α is updated to identify operations that generalize well to unseen traffic patterns.

Role of α . Each edge in a cell's DAG is relaxed into a weighted combination of candidate primitives:

$$o^{(i,j)}(x) = \sum_{o \in O} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in O} \exp(\alpha_{o'}^{(i,j)})} \cdot o(x),$$

allowing α to be optimized via gradient descent alongside w . Since α is updated with gradients from the validation set, the architecture is guided toward operations that generalize across traces, while the training set ensures the weights learn temporal/directional patterns.

Model augmentation phase. Once the search phase concludes, the continuous architecture parameters α encode the relative importance of each primitive operation on every edge of a cell's DAG. To instantiate an architecture suitable for WF, we follow the standard cell discretization procedure used by DARTS [50].

DARTS cell discretization. For each cell's intermediate node, we retain the two strongest incoming edges (by highest architecture weight α , after excluding the *none* operation). For each selected edge, we keep only the single primitive operation with the highest weight. Thus, each node aggregates the outputs of two incoming edges, each associated with one selected operation. Formally, let (i_1, j) and (i_2, j) denote the two selected incoming edges for node j , and let $o_{i_1 \rightarrow j}$ and $o_{i_2 \rightarrow j}$ denote their corresponding highest-weight operations. The node output is then computed as the sum of these two operations: $h_j(x) = o_{i_1 \rightarrow j}(x) + o_{i_2 \rightarrow j}(x)$.

Final architecture definition. The cell structures resulting from discretization are stacked to form the final CRICKET architecture, which consists of n cells arranged hierarchically (see §2.3). In our experimental setup (see §4.2), we defined such an architecture with $n = 4$ cells. This final architecture begins with a *stem* (the layers that process the raw input data before it enters the cells that are subject to architecture search [50]) consisting of two convolutional layers with batch normalization and ReLU activations that project the raw two-channel input signal into 108 feature channels. The main body of the model comprises four stacked cells: two normal cells and two reduction cells. Each cell contains four intermediate nodes, and each node aggregates two operations, resulting in a total of eight convolutional operations per cell. Following the four cells, the network applies a global average pooling layer to reduce the temporal dimension to one, and the last fully connected layer maps the resulting 576-dimensional feature vector to output classes. For more details on this final architecture, we direct the reader to a complete description and illustration of its structure in Appendix A.

Final architecture training. The resulting stacked architecture is then trained, validated, and tested from scratch using an 80/10/10 split (as also suggested by Liu et al. [50]). Here, the α parameters gathered from the model search phase have already been discarded, so that only the final discrete-cell architecture is retained.

4 Evaluation Methodology

4.1 Evaluation Goals and Metrics

Our evaluation (results discussed later in §5) is focused on gauging CRICKET's ability to address each of the following primary goals:

Efficacy. First, to assess the effectiveness of CRICKET at identifying website visits over Tor, we evaluate its classification performance on both the closed- and open-world WF setting. As metrics for closed-world experiments, we report classification *accuracy*, reflecting an adversary's ability to correctly identify traces pertaining to a monitored set of websites. For our open-world experiments, we report CRICKET's *precision*, *recall*, and *F1 score*. These scores are computed as support-weighted averages across all classes, including the monitored website classes and the additional unmonitored class.

Robustness. Second, we evaluate whether the neural architecture search used by CRICKET remains effective across varying conditions, including WF defenses applied to Tor traffic, different Tor trace datasets, and variations in the search process itself. Specifically, we

measure CRICKET’s efficacy on defended traffic (using the metrics defined above) and experiment with the transferability of architectures discovered via DARTS by mixing and matching training and testing across Tor datasets. We also perform a sensitivity analysis over random seeds and training hyperparameters, assessing the stability and consistency of the discovered architectures.

Practicality. Third, we gauge whether CRICKET is practical in real-world deployments. This encompasses measuring the size of the models produced by our attack pipeline (i.e., number of *trainable parameters*), and the computational overhead imposed by architecture searches (in *wall-clock time* and *hardware resource usage*).

4.2 Experimental Testbed

We now describe our laboratory testbed and CRICKET’s NAS setup.

Hardware. All experiments were conducted on a server equipped with a single NVIDIA H100 GPU (80 GB HBM3 memory), dual Intel Xeon Platinum 8480C CPUs (112 cores total), and 2 TB of RAM.

Software. The server deploying CRICKET ran on Ubuntu 24.04.1 LTS. We used CUDA 12.6, cuDNN 9.6, and PyTorch 2.7.0, running on Python 3.12.3. Random seeds were fixed in NumPy and PyTorch.

DARTS model search/augmentation setup. The number of stacked cells in the DARTS search network is a user-specified hyperparameter that helps balance the expressivity of the final model and the search process overhead. While the original DARTS setup uses eight cells in its supernet [50], we opted for a smaller search network with four cells ($n = 4$); prior work unveiled that shallow supernet networks provide sufficient depth for candidate operations’ evaluation while keeping the search tractable [17, 94].

DARTS computational budget. CRICKET’s DARTS backbone also assumes a pre-defined computation budget for exploration. In line with the number of training epochs assumed by previous work (e.g., Tik-Tok [69]), we configured DARTS’ model search phase and subsequent model augmentation phase to train for 30 epochs each.

DARTS parameters. We relied on DARTS’ default parameters and did not optimize any of its algorithm-specific hyperparameters exposed during either the search or augment phases throughout our evaluation (yet, ablation results are later discussed in §5.7).

4.3 Datasets

Our evaluation relies on three datasets that span synthetic benchmarks, realistic Tor traffic, and large-scale open-world settings.

Traces. We leverage two main families of Tor traces (see Appendix B for a rundown of ethical considerations regarding the use of these traces) to develop suitable datasets for CRICKET’s evaluation: *AWF*. The AWF [72] family of datasets was constructed using automated crawling and instrumentation to collect network traces from repeated visits to a set of monitored websites; such synthetic collections have been used as canonical benchmarks for WF attack and defense development and comparison. In this work, we use a subset denoted by AWF_{100} , comprising 100 visits to each of 100 different monitored websites (10,000 traces total).

GTT. Moving away from synthetic website crawls, recent efforts have aimed to collect datasets of genuine Tor traffic exchanged by real users. This traffic includes interactions with any services accessible via the internet (including non-web services), allowing

for the identification of websites browsed through Tor based on first-party domain names. Genuine data better reflects Tor variability (circuit-length variance, base rates, and natural congestion), being therefore more challenging and realistic for evaluation. We use two subsets derived from the GTT [43] dataset:

GTT₁₀₀: a subset of GTT with 100 traces for each of 100 monitored domains (10,000 traces total). It exhibits genuine traffic characteristics while matching the number of websites/traces of AWF_{100} .

GTT₂₀₀: a larger-scale subset of GTT containing 200 monitored domains with 2,000 traces each (400,000 traces total) used to stress-test the efficacy, robustness and practicality of CRICKET. It also grounds later closed- and open-world comparisons.

OpenGTT₂₀₀: This subset, employed for open-world experiments (§5.3), comprises the same 400,000 traces included in GTT_{200} (monitored set), but includes an additional 400,000 traces, each sourced from a different domain (unmonitored set). As each unmonitored trace originates from a distinct domain, unmonitored domains are disjoint across the train/validation/test sets used for evaluation.

Trace pre-processing. For consistency, we truncate or zero-pad all traces to a fixed-length cell sequence comprising 5,000 cells. The feature sets used by the WF attacks considered in our experiments are extracted from this uniform representation.

4.4 Baseline Attacks and Defenses

Attacks. We compare the effectiveness of CRICKET against four SOTA WF attacks whose DNNs’ span the features described in §3.2:

Deep Fingerprinting (DF) [80]: A CNN that ingests cells’ directional information. We also adapt DF to our *3-channel representation* (§3.2).

Tik-Tok [69]: Architecturally identical to DF, Tik-Tok consumes the raw packet trace as a 1D directional+timing sequence. Similarly to DF, we also adapt Tik-Tok to our three-channel representation.

Robust Fingerprinting (RF) [75]: An ensemble-learning approach that uses TAM as a single-channel input vector. Each trace is aggregated into temporal bins, with the bins’ cell counts serving as features.

Laserbeak [54]: A hybrid architecture combining convolutional layers and transformer encoders for modeling long-range dependencies. Its inputs relies on a six-channel feature representation.

For fairness, we adopt the proposed input representation for each baseline (e.g., TAM for RF, engineered features for Laserbeak) and retrain the models under identical conditions on our datasets. For DF and Tik-Tok, which operate on raw directional and directional+timing traces, we use one- and three-channel encodings.

Defenses. We further protect the undefended traces via five representative WF defenses (used in recent WF attacks’ evaluation (e.g., [24, 75]) that cover standard padding-based strategies: fixed-rate (*CS-BuFLO* [9], *Tamaraw* [10]), adaptive padding (*WTF-PAD* [47]), randomized early-padding (*FRONT* [31]), and regulation (*Regula-Tor* [40]). We use all defenses with their standard parameters, except for *FRONT*, for which we use Veicht et al. [83]’s T1 and T2 variants.

5 Evaluation Results

We now discuss the results of CRICKET’s evaluation. We first assess the benefits of an increased number of channels in the direction(+timing) input representation for Tor cells (§5.1). Then, we compare CRICKET against SOTA WF attacks in the closed- (§5.2) and

Table 3: Multi-class closed-world classification accuracy when varying the cell encoding method.

Encoding	AWF ₁₀₀		GTT ₁₀₀	
	DF	Tik-Tok	DF	Tik-Tok
1-channel	96.2±0.23	95.7±0.24	60.2±0.5	22.0±2.8
3-channel	96.3±0.29	95.7±0.26	60.0±0.46	29.1±2.7

open-world (§5.3) settings, and against defended traces (§5.4). We gauge the transferability of CRICKET-searched models across WF datasets (§5.5 and §5.6). Lastly, we assess the stability of CRICKET results and quantify the impact of NAS in its analysis pipeline (§5.7). Appendix C details our plan for releasing CRICKET’s source code and the datasets supporting this section’s experiments.

Search and augmentation. Throughout our evaluation, we highlight CRICKET’s deployment across different feature representations and datasets. Following the standard DARTS workflow, which consists of a search phase and an augment phase, we started by performing a full architecture search separately for each feature set on undefended Tor traces (§5.2). Later, when evaluating attack efficacy in the open-world (§5.3) and against defenses (§5.4), we opt not to re-run the costly search procedure, leveraging instead the architectures discovered on undefended traces, limiting ourselves to execute the augment phase. This approach allows us to study the adaptability and robustness of CRICKET across these scenarios while significantly reducing computational overhead tied to architecture search. Finally, for completeness and to explore the impact of architecture search on the efficacy of CRICKET against defended traces, we perform a set of experiments where we conduct both search and augmentation directly on defended traces (§5.6).

5.1 Impact of One- vs. Three-Channel Inputs

We first evaluate our three-channel input representation, which improves on the standard one-channel encoding by disentangling the cell signal used for directions and padding to prevent the model from conflating different traffic phenomena (see §3.2).

Table 3 reports the closed-world classification accuracy of the CNN models operating on directional (DF) and directional+timing (Tik-Tok) data when trained on AWF₁₀₀ and GTT₁₀₀ datasets under both encoding schemes. Our results suggest that, while the three-channel encoding provides little to no benefit when targeting AWF₁₀₀ (classification accuracy remaining essentially the same), this representation allow Tik-Tok to achieve an improvement of ~7 percentage points (pp) on GTT₁₀₀ traces. This indicates that when more useful information is available (Tik-Tok exploits timing whereas DF does not), our 3-channel representation allows the classifier to better utilize that information to improve accuracy. Thus, in the remainder of our experiments, we use our three-channel representation when comparing CRICKET with DF and Tik-Tok.

5.2 CRICKET in the Closed-World Setting

Armed with the insight that a three-channel input representation may benefit a model that leverages the directional+timing representation, we now more thoroughly investigate how different feature representations influence model efficacy and practicality. To this end, we compare CRICKET-searched model architectures against our set of baseline WF attacks under four widely adopted input

Table 4: Attack model sizes and closed-world accuracy for multiple feature representations (undefended traces).

Input Format	Attack	Params. (Millions)	Accuracy (%)		
			AWF ₁₀₀	GTT ₁₀₀	GTT ₂₀₀
Direction+Timing (3-channels)	Tik-Tok	3.96	95.96	31.75	57.90
	CRICKET	1.90	94.50	38.80	62.79
Direction (3-channels)	DF	3.96	96.59	61.00	74.14
	CRICKET	1.90	96.65	65.00	75.40
TAM Vector	RF	1.01	93.60	58.50	78.31
	CRICKET	1.90	94.13	62.40	78.94
Laserbeak (6-channels)	Laserbeak	4.78	96.20	61.70	79.63
	CRICKET	1.92	95.30	67.32	82.20

formats: i) cell direction, ii) cell direction+timing, iii) TAM vectors, and iv) Laserbeak’s engineered representation. For each representation, we report closed-world accuracy on AWF₁₀₀, GTT₁₀₀, and GTT₂₀₀ (for measuring efficacy), along with the total parameter count (for gauging practicality). Our results are shown in Table 4.

Timing is not beneficial with our 3-channel representation. Our first observation is that directional+timing features seem detrimental to classification performance in comparison to direction only. For instance, on AWF₁₀₀, we observe that CRICKET obtained ~2 pp higher accuracy when using direction-only data (96.65%) vs. direction+timing (94.50%). This increase in efficacy is also noticeable when comparing the original attacks leveraging each different representation, with DF surpassing Tik-Tok by ~1 pp.

For GTT₁₀₀, CRICKET achieves an accuracy of 65.00% with directional inputs, whereas it achieves only 38.80% accuracy when considering directional+timing inputs. This trend is also observed when comparing DF and Tik-Tok across representations. While the gap between representations gets closer on GTT₂₀₀ dataset, CRICKET still achieves ~13 pp higher accuracy with direction-only inputs.

CRICKET outperforms attacks using direction-based representations. For the *direction+timing* and *direction-only* representations, CRICKET outperforms their CNN counterparts (Tik-Tok and DF). The gain is most evident on GTT₁₀₀ and GTT₂₀₀, where we find CRICKET improves accuracy by +7.0 pp and +1.3 pp (direction+timing), and by +4.0 pp and +1.3 pp (direction-only). This showcases the ability of NAS to automatically adapt architectures to 1D packet sequences more effectively than hand-designed CNNs.

Engineered features increase CRICKET’s accuracy. The TAM representation remains a strong baseline, with RF achieving competitive results using only ~1M parameters. Still, CRICKET achieves higher accuracy across all datasets. When equipped with Laserbeak’s features, CRICKET achieves its best overall efficacy, surpassing the original Laserbeak model on GTT₁₀₀ (+5.6 pp) and GTT₂₀₀ (+2.6 pp), while using a model less than half the size (1.94M vs. 4.78M parameters). Though slightly underperforming on AWF₁₀₀ (-0.9 pp), the trend shows that NAS-based architectures can improve upon hybrid CNN–Transformer pipelines using the same features. **CRICKET produces compact models.** Across all input formats, CRICKET produced significantly more compact DNN models than all but RF while preserving or exceeding the SOTA accuracy. CRICKET produced models with fewer than 2M parameters on average, which is about 1/3 to 1/2 as many parameters as is used by Tik-Tok, DF, and Laserbeak. Such compactness is desirable in adversarial contexts, where smaller models are cheaper to train and deploy at scale.

Table 5: Open-world attack efficacy on OpenGTT₂₀₀ (*set-64*). CRICKET matches or outperforms SOTA attacks.

Input Format	Attack	Precision	Recall	F1-score
Direction+Timing (3-channels)	Tik-Tok	0.8505	0.8494	0.8437
	CRICKET	0.8776	0.8823	0.8755
Direction (3-channels)	DF	0.8694	0.8707	0.8680
	CRICKET	0.8691	0.8726	0.8666
TAM Vector	RF	0.8834	0.8849	0.8808
	CRICKET	0.8799	0.8829	0.8787
Laserbeak (6-channel)	Laserbeak	0.8992	0.8975	0.8964
	CRICKET	0.9057	0.9077	0.9055

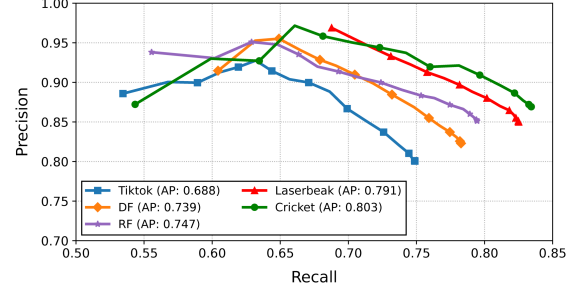
CRICKET’s computational overhead is tractable. On GTT₁₀₀ (10k traces, TAM representation), the search phase (30 epochs, batch size 64, 8-layer supernet) takes 1h32m on an NVIDIA H100 (80 GB), with peak memory usage of ~63 GB. When scaling to GTT₂₀₀ (400k traces), search time increases to 1d9h33m but memory usage remains stable. This indicates that the dominant factor impacting runtime is dataset scale, whereas memory usage is dictated by the supernet size rather than the number of traces.

5.3 CRICKET in the Open-World Setting

We now examine whether our previous findings translate to the more realistic open-world scenario. Following the methodology of Shen et al. [75], we cast the open-world setting as a more challenging multi-class classification task rather than a binary (monitored vs. unmonitored) decision problem. Thus, we treat unmonitored sites as an additional class during training, allowing the classifier to learn features that distinguish monitored from unmonitored sites.

We rely on the same final architectures discovered by CRICKET during its model search phase (across each different input representation) focused on the closed-world setting (§5.2). Thus, when dealing with open-world data, we simply conduct a model augment phase that retrains CRICKET’s final architectures on multiple monitored website classes and a wider unmonitored class. To measure attack efficacy as a function of the size of the considered monitored set of websites, we gradually increase the number k of sites included in the monitored set (*set-k*), where $k \in \{16, 32, 64, 128, 200\}$. For each experiment, the unmonitored class is intentionally unbalanced, containing a total number of samples equal to the combined samples of all monitored classes in the corresponding *set-k* setting. **CRICKET outperforms existing attacks.** In our first experiment, we sought to understand the efficacy of CRICKET in the open-world scenario when targeting a particular *set-k* value. We selected $k = 64$, which is the center of our considered range. Table 5 shows the efficacy of different attacks and feature set combinations on the *set-64* configuration of the OpenGTT₂₀₀ dataset. The results reported in this table suggest that CRICKET consistently matches or outperforms other attack models across all feature families.

For a closer comparison, we show the precision-recall (PR) curves for the different attacks in Figure 5. Each point on a PR curve corresponds to a decision threshold ranging from $[0, 1]$ in intervals of 0.05, capturing how model confidence influences detection trade-offs. Here, the monitored set represents the classes under observation; if the model’s predicted probability for a monitored class falls below a given threshold, the instance is marked as unmonitored.

**Figure 5: Precision-recall curves for CRICKET and Laserbeak in the open-world (*set-64*) setting.****Table 6: Open-world attack efficacy (F1 score) on OpenGTT₂₀₀ considering different monitored *set-k* sizes.**

Attack	F1 score for monitored <i>set-k</i> size				
	Set-16	Set-32	Set-64	Set-128	Set-200
CRICKET	0.9431	0.9215	0.9055	0.8938	0.8618
Laserbeak	0.9250	0.9051	0.8964	0.8925	0.8624

When paired with Laserbeak’s features, CRICKET achieves the highest average precision (AP), outperforming DF, Tik-Tok, RF, and Laserbeak. Despite usually having 1/3 to 1/2 as many parameters as the baseline models, CRICKET exhibits the highest precision across all models, up to a threshold of 0.5 (corresponding to the peak precision of around 0.97 with 0.66 recall). The decline in performance beyond this point can be attributed to the smaller model size, which may lead to reduced confidence at higher thresholds. This is consistent with trends observed in other compact models such as RF, where reduced model capacity results in a diminished efficacy.

Laserbeak features remain the most informative in the open-world setting. Table 5 shows that Laserbeak’s features are again most effective: CRICKET achieves 0.9055 F1-score compared to 0.8964 attained by the original Laserbeak attack. Recall that the final architecture found earlier by CRICKET when using Laserbeak’s features (§5.2) uses fewer than half the parameters of the Laserbeak attack (CRICKET’s 1.92M is 40% of Laserbeak’s 4.78M parameters).

CRICKET is more effective across all *set-k* sizes. Table 6 shows that CRICKET (w/ Laserbeak’s features) attains a higher F1-score than Laserbeak across most monitored *set-k* sizes, with margins widening at smaller monitored sets (e.g., +1.81 pp at *set-16*). CRICKET’s advantage decreases as the size of the monitored set increases.

5.4 Playing CRICKET against Defenses

To assess CRICKET’s robustness, we conducted a set of experiments where we target defended Tor traffic. We selected the most effective input representation (i.e., Laserbeak’s engineered features) found when targeting undefended traces. As in §5.3, we execute CRICKET’s model augment phase using defended traces (produced by each of the different defenses we consider) and the 1.92M parameter final architecture discovered for undefended traces leveraging Laserbeak’s representation—see Table 4 in §5.2. We benchmark the efficacy of CRICKET against Laserbeak when targeting the defenses described in §4.4, evaluating attack efficacy in the closed world (AWF₁₀₀, GTT₁₀₀, GTT₂₀₀) and open world (OpenGTT₂₀₀ with *set-64*).

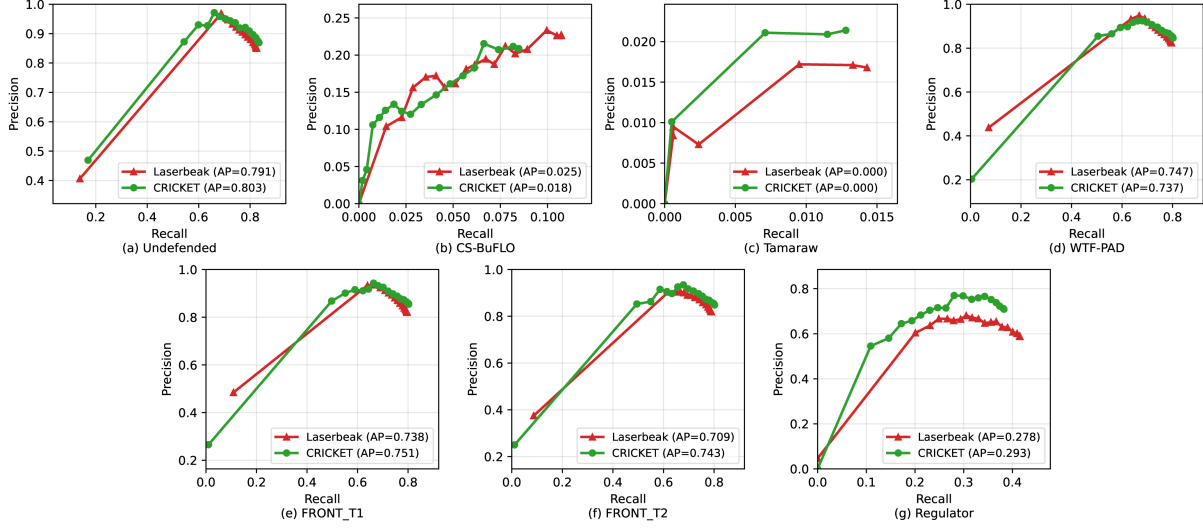


Figure 6: Precision-recall curves for CRICKET and Laserbeak in the open-world setting.

Table 7: Laserbeak’s and CRICKET’s efficacy on closed-world datasets against different defenses.

Defense	Attack	Accuracy (%)		
		AWF ₁₀₀	GTT ₁₀₀	GTT ₂₀₀
CS-BuFLO	Laserbeak	11.40	16.00	17.82
	CRICKET	11.00	14.00	17.18
Tamaraw	Laserbeak	4.30	4.80	3.90
	CRICKET	4.54	4.70	4.06
WTF-PAD	Laserbeak	89.20	50.80	77.16
	CRICKET	89.90	62.50	79.01
FRONT_T1	Laserbeak	87.40	53.10	77.05
	CRICKET	87.30	64.20	79.44
FRONT_T2	Laserbeak	80.90	50.30	76.51
	CRICKET	87.80	63.40	78.94
RegulaTor	Laserbeak	23.80	25.80	39.86
	CRICKET	27.00	27.20	44.17

Closed-world. Table 7 summarizes our results on closed-world defended Tor traffic. CRICKET’s accuracy matches or surpasses Laserbeak’s against defenses such as WTF-PAD, FRONT, and RegulaTor. CRICKET’s accuracy gains are most pronounced against WTF-PAD and FRONT_T2, where CRICKET improves accuracy vs. Laserbeak by up to +13 pp on GTT₁₀₀. Lastly, we see that both CRICKET and Laserbeak struggle under strong obfuscation schemes such as Tamaraw and CS-BuFLO, which enforce rigid regularity of network traces.

Open-world. Table 8 summarizes open-world experiments on defended Tor traffic using OpenGTT₂₀₀ (set-64). Our results are broadly consistent with the trends found in closed-world settings. For instance, under WTF-PAD and FRONT_T2, CRICKET improves F1 score vs. Laserbeak by +1.1 pp and +1.6 pp, respectively. Like before, CS-BuFLO and Tamaraw greatly reduce attack efficacy.

Lastly, Figure 6 plots the PR curves of CRICKET and Laserbeak when targeting defended traces. CRICKET achieves a higher average precision (AP) than Laserbeak when facing most defended traces (with the exception of CS-BuFLO and Tamaraw, where both models perform poorly), despite using only 40% as many parameters.

Table 8: Comparison of Laserbeak’s and CRICKET’s efficacy on OpenGTT₂₀₀ (set-64) against different defenses.

Defense	Attack	Efficacy on OpenGTT ₂₀₀ (set-64)		
		Precision	Recall	F1 Score
CS-BuFLO	Laserbeak	0.1432	0.1570	0.1404
	CRICKET	0.1370	0.1529	0.1394
Tamaraw	Laserbeak	0.0349	0.0585	0.0436
	CRICKET	0.0351	0.0585	0.0435
WTF-PAD	Laserbeak	0.8777	0.8776	0.8752
	CRICKET	0.8852	0.8893	0.8849
FRONT_T1	Laserbeak	0.8777	0.8786	0.8762
	CRICKET	0.8884	0.8911	0.8867
FRONT_T2	Laserbeak	0.8740	0.8740	0.8711
	CRICKET	0.8865	0.8891	0.8850
RegulaTor	Laserbeak	0.6491	0.6739	0.6340
	CRICKET	0.6826	0.6816	0.6239

5.5 Assessing CRICKET’s Transferability

NAS is a computationally expensive process, even when employing efficient methods such as DARTS. Thus, identifying strategies that yield architectures with strong generalization properties is particularly valuable. Intuitively, performing model searches on challenging datasets (e.g., GTT₁₀₀’s genuine traces) may lead to the discovery of more expressive architectures that can effectively transfer to simpler datasets (e.g., AWF₁₀₀’s synthetic traces).

To test this hypothesis, we employed CRICKET to search on both AWF₁₀₀ (synthetic) and GTT₁₀₀ (genuine), across three different input representations (direction with 3 channels, TAM, and Laserbeak’s features) and then cross-evaluated the resulting architectures. The results of this experiment, shown in Table 9, support our hypothesis: architectures searched on the more realistic GTT₁₀₀ dataset exhibit an improved transferability when retrained and evaluated on AWF₁₀₀. In contrast, architectures discovered on AWF₁₀₀ fail to generalize as effectively to GTT₁₀₀. This pattern holds consistently across all three input representations.

Table 9: Efficacy of CRICKET-searched architectures with cross-dataset training on AWF₁₀₀ and GTT₁₀₀.

Search Dataset	Test Dataset	Accuracy (%) across feature sets		
		Direction (3-ch)	TAM	Laserbeak
AWF ₁₀₀	AWF ₁₀₀	90.70	91.20	93.90
GTT ₁₀₀	AWF ₁₀₀	96.65	94.13	95.30
AWF ₁₀₀	GTT ₁₀₀	58.30	53.10	66.20
GTT ₁₀₀	GTT ₁₀₀	65.00	62.40	67.32

5.6 Searching Architectures on Defended Traces

We now extend our search process to operate over close-world defended traces and analyze the efficacy of the resulting architectures. **Search/augmentation setup.** Instead of performing a separate search for every dataset/defense pair, we conduct searches exclusively on GTT₁₀₀ for each defense and then simply retrain (augment) the discovered architectures on both AWF₁₀₀ and GTT₁₀₀. Building on §5.5 findings, this reduces computational costs and exploits the transferability benefits of searching on genuine traces.

Searching on defended traces boosts efficacy. The results in Table 10 show the effectiveness of the model search + augment strategy. In all but one of the defense+dataset combinations we considered, the architectures CRICKET discovered when searching on defended GTT traces (*defended-search*) outperform those discovered when searching on undefended GTT traces (*undefended-search*) when testing on defended AWF₁₀₀ and GTT₁₀₀ traces. For example, for FRONT_T2, undefended-search achieves an accuracy of 63.4% on GTT₁₀₀ (from Table 10), whereas defended-search accuracy improves by 1.5 pp despite using a smaller model (1.41M vs. 1.92M parameters). For RegulaTor, undefended-search achieves an accuracy of 27.2% on GTT₁₀₀ (from Table 10), whereas defended-search accuracy improves by 2.1 pp again using a smaller model (0.75M vs. 1.92M parameters). Further, in defenses where undefended-search previously underperformed when compared to Laserbeak—such as CS-BuFLO & Tamaraw—our new defended-search architectures improve CRICKET’s accuracy by +2 pp and +0.3 pp, respectively. Defended-search architectures also vary structurally, suggesting that the search adapts its capacity to each defense’s traffic pattern.

5.7 Stability Analysis and NAS Ablation

Stability and hyperparameter sensitivity. We assess the robustness of CRICKET’s search to variations in random seeds and training hyperparameters on GTT₁₀₀ using Laserbeak’s features, finding it stable and consistently yielding high-performing architectures.

Random seeds. Searches with four random seeds resulted in models achieving accuracies of 65.50%, 67.00%, 66.90%, and 67.10% (mean 66.63% $\pm$ 0.66 pp), compared to 67.32% for the original genotype. Discovered architectures exhibit slight variations in structure and parameter counts (0.68M–0.97M), but performance is consistent.

Hyperparameters. We varied DARTS’ weight (w_{lr}) and architecture (α_{lr}) learning rates. Varying w_{lr} among {0.0125, 0.0375} (default: 0.025) yields accuracies of 67.20% (-0.12 pp) and 65.40% (-1.92 pp). Varying α_{lr} among $\{2 \times 10^{-4}, 4 \times 10^{-4}\}$ (default: 3×10^{-4}) yields accuracies of 67.21% (-0.11 pp) and 66.70% (-0.62 pp). We find that the discovered architectures vary in size across all configurations (0.84M–1.14M parameters), with no clear correlation with accuracy.

Table 10: CRICKET’s closed-world efficacy with defended-search (compared to undefended-search results in Table 7).

Legend: defended-search wins, undefended-search wins, and CRICKET’s defended-search model outperforms Laserbeak.

Defense	Params. (Millions)	Accuracy (%)	
		AWF ₁₀₀	GTT ₁₀₀
CS-BuFLO	1.16	13.30 (+2.30 pp)	16.00 (+2.00 pp)
Tamaraw	1.16	5.32 (+0.78 pp)	5.00 (+0.30 pp)
WTF-PAD	1.64	89.30 (-0.60 pp)	62.69 (+0.19 pp)
FRONT_T1	1.41	87.43 (+0.13 pp)	64.70 (+0.50 pp)
FRONT_T2	1.41	88.33 (+0.53 pp)	64.90 (+1.50 pp)
RegulaTor	0.75	31.40 (+4.40 pp)	29.30 (+2.10 pp)

Ablation to assess NAS’ impact. To quantify the contribution of CRICKET’s NAS, we compare the searched genotype against randomized architectures on GTT₁₀₀ using Laserbeak’s features.

Capacity-matched randomization. We generated three random genotypes by shuffling primitive placements within the discovered cell topology while preserving convolutional operators and model parameters (~ 1.98 – 2.15 M). These models achieved accuracies of 65.6%, 66.6%, and 66.5% (mean 66.23% $\pm$ 0.55 pp) vs. CRICKET’s 67.32%.

Full-pool randomization. We generated three genotypes by uniformly sampling from the full primitive pool. These models achieved accuracies of only 63.2%, 64.6%, and 66.4% (mean 64.73% $\pm$ 1.60 pp), with lower capacity (~ 1.10 – 1.34 M params.). Naive sampling is prone to select suboptimal operators, yielding subpar performance.

6 Conclusion

This paper introduces CRICKET, a framework that automates the search for neural architectures suited to website fingerprinting attacks on Tor. By adapting the DARTS neural architecture search method to handle 1D Tor cell traces, CRICKET discovers compact DNN models that outperform or match the efficacy of SOTA attacks on both synthetic and genuine Tor datasets. Our results suggest that automated DNN exploration can yield more efficient and effective WF models when compared to those obtained via domain experts’ heuristic choices, especially on real-world, genuine traces.

Future work. Pursuing a set of complementary research avenues can broaden the impact of CRICKET. We highlight two directions:

Advanced NAS substrates. Although DARTS offers an efficient search procedure, it is sensitive to search hyperparameters [97]. Future work can explore more sophisticated NAS frameworks that offer increased stability and generalizability, such as PC-DARTS [95] and DrNAS [12], as well as alternative paradigms: e.g., evolutionary search [70] and reinforcement learning-based NAS [66, 102, 103].

Expanding WF settings. Extending CRICKET to overlapping streams or early detection from partial traces would “stress-test” how well NAS-discovered architectures generalize across traffic patterns.

Acknowledgments

This work was partially supported by the U.S. Naval Research Laboratory Base Program and by NSERC under grant RGPIN-2023-03304. We thank the Thapar School of Advanced AI and Data Science at TIET, Patiala for their compute resources. We thank Shreya Arun Naik and Mazen Khodier for helpful discussions about this work.

References

- [1] Kota Abe and Shigeki Goto. 2016. Fingerprinting attack on Tor anonymity using deep learning. *Proceedings of the Asia-Pacific Advanced Network* 42 (2016), 15–20.
- [2] Alireza Bahramali, Ardavan Bozorgi, and Amir Houmansadr. 2023. Realistic Website Fingerprinting By Augmenting Network Traces. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [3] Bowen Baker, Otakrist Gupta, Ramesh Raskar, and Nikhil Naik. 2017. Accelerating neural architecture search using performance prediction. *arXiv preprint arXiv:1705.10823* (2017).
- [4] Gabriel Bender, Pieter-Jan Kindermans, Barret Zoph, Vijay Vasudevan, and Quoc Le. 2018. Understanding and simplifying one-shot architecture search. In *Proceedings of the International Conference on Machine Learning*. 550–559.
- [5] Gabriel Bender, Hanxiao Liu, Bo Chen, Grace Chu, Shuyang Cheng, Pieter-Jan Kindermans, and Quoc V Le. 2020. Can weight sharing outperform random architecture search? An investigation with TuNAS. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 14323–14332.
- [6] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for hyper-parameter optimization. *Advances in Neural Information Processing Systems* 24 (2011), 2546–2554.
- [7] Sanjit Bhat, David Lu, Albert Kwon, and Srinivas Devadas. 2019. Var-CNN: A data-efficient website fingerprinting attack based on deep learning. In *Proceedings on Privacy Enhancing Technologies*, Vol. 4. 292–310.
- [8] Han Cai, Tianyao Chen, Weinan Zhang, Yong Yu, and Jun Wang. 2018. Efficient architecture search by network transformation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [9] Xiang Cai, Rishabh Nithyanand, and Rob Johnson. 2014. CS-BuFLO: A congestion sensitive website fingerprinting defense. In *Proceedings of the 13th ACM Workshop on Privacy in the Electronic Society*. 121–130.
- [10] Xiang Cai, Rishabh Nithyanand, Tao Wang, Rob Johnson, and Ian Goldberg. 2014. A systematic approach to developing and evaluating website fingerprinting defenses. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 227–238.
- [11] Xiang Cai, Xin Cheng Zhang, Brijesh Joshi, and Rob Johnson. 2012. Touching from a distance: Website fingerprinting attacks and defenses. In *Proceedings of the 2012 ACM SIGSAC Conference on Computer and Communications Security*. 605–616.
- [12] Xiangning Chen, Ruochen Wang, Minhao Cheng, Xiaocheng Tang, and Cho-Jui Hsieh. 2021. DrNAS: Dirichlet Neural Architecture Search. In *Proceedings of the 9th International Conference on Learning Representations*.
- [13] Yanguang Chen. 2023. Spatial autocorrelation equation based on Moran's index. *Scientific reports* 13, 1 (2023), 12996.
- [14] Giovanni Cherubin, Jamie Hayes, and Marc Juárez. 2017. Website Fingerprinting Defenses at the Application Layer. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2017, issue 2. 186–203. Issue 2.
- [15] Giovanni Cherubin, Rob Jansen, and Carmela Troncoso. 2022. Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world. In *31st USENIX Security Symposium (USENIX Security 22)*. 753–770.
- [16] Patryk Chrabaszcz, Ilya Loshchilov, and Frank Hutter. 2017. A downsampled variant of ImageNet as an alternative to the CIFAR datasets. *arXiv preprint arXiv:1707.08819* (2017).
- [17] Xiangxiang Chu, Tianbao Zhou, Bo Zhang, and Jixiang Li. 2020. Fair darts: Eliminating unfair advantages in differentiable architecture search. In *European conference on computer vision*. Springer, 465–480.
- [18] Yuwen Cui, Guangjing Wang, Khanh Vu, Kai Wei, Kehan Shen, Zhengyuan Jiang, Xiao Han, Ning Wang, Zhuo Lu, and Yao Liu. 2025. A Comprehensive Survey of Website Fingerprinting Attacks and Defenses in Tor: Advances and Open Challenges. *arXiv:2510.11804 [cs.CR]* *arXiv:2510.11804*.
- [19] Wladimir De la Cadena, Asya Mitseva, Jens Hiller, Jan Pennekamp, Sebastian Reuter, Julian Filter, Thomas Engel, Klaus Wehrle, and Andriy Panchenko. 2020. Trafficsliver: Fighting website fingerprinting attacks with traffic splitting. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1971–1985.
- [20] Xinhao Deng. [n. d.]. Website-Fingerprinting-Library: A Library for Advanced DL-based Website Fingerprinting Attacks. <https://github.com/Xinhao-Deng/Website-Fingerprinting-Library>. Accessed: 2026-07-16.
- [21] Xinhao Deng, Qi Li, and Ke Xu. 2024. Robust and reliable early-stage website fingerprinting attacks via spatial-temporal distribution analysis. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 1997–2011.
- [22] Xinhao Deng, Qilei Yin, Zhuotao Liu, Xiyuan Zhao, Qi Li, Mingwei Xu, Ke Xu, and Jianping Wu. 2023. Robust multi-tab website fingerprinting attacks in the wild. In *Proceedings of the 44th IEEE Symposium on Security and Privacy*. 1005–1022.
- [23] Xinhao Deng, Yixiang Zhang, Qi Li, Zhuotao Liu, Yabo Wang, and Ke Xu. 2026. Enhancing Website Fingerprinting Attacks against Traffic Drift. In *Proceedings of the 33rd Network and Distributed System Security Symposium*.
- [24] Xianwen Deng, Ruijie Zhao, Yanhao Wang, Mingwei Zhan, Zhi Xue, and Yijun Wang. 2025. Countmamba: A Generalized Website Fingerprinting Attack via Coarse-Grained Representation and Fine-Grained Prediction. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1419–1437.
- [25] Roger Dingledine, Nick Mathewson, Paul F Syverson, et al. 2004. Tor: The second-generation onion router. In *Proceedings of the 13th USENIX Security Symposium*, Vol. 4. 303–320.
- [26] Priyanka Dodia, Mashael AlSabah, Omar Alrawi, and Tao Wang. 2022. Exposing the Rat in the Tunnel: Using Traffic Analysis for Tor-Based Malware Detection. In *Proceedings of the 2022 SIGSAC Conference on Computer and Communications Security*. 875–889.
- [27] Kevin P Dyer, Scott E Coull, Thomas Ristenpart, and Thomas Shrimpton. 2012. Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail. In *Proceedings of the 33rd IEEE Symposium on Security and Privacy*. 332–346.
- [28] Thomas Elsken, Jan-Hendrik Metzen, and Frank Hutter. 2017. Simple and efficient architecture search for convolutional neural networks. *arXiv preprint arXiv:1711.04528* (2017).
- [29] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. 2019. Neural architecture search: A survey. *The Journal of Machine Learning Research* 20, 1 (2019), 1997–2017.
- [30] Peter I Frazier. 2018. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811* (2018).
- [31] Jiajun Gong and Tao Wang. 2020. Zero-delay lightweight defenses against website fingerprinting. In *Proceedings of the 29th USENIX Security Symposium*. 717–734.
- [32] Jiajun Gong, Wuqi Zhang, Charles Zhang, and Tao Wang. 2022. Surakav: Generating realistic traces for a strong website fingerprinting defense. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy*. 1558–1573.
- [33] Sonia Gu. 2022. *Leveraging Automated Machine Learning for Website Fingerprinting*. Master's thesis. Princeton University - Electrical and Computer Engineering.
- [34] Ankit Gupta and Alexander M Rush. 2017. Dilated convolutions for modeling long-distance genomic dependencies. *arXiv preprint arXiv:1710.01278* (2017).
- [35] Jamie Hayes and George Danezis. 2016. k-fingerprinting: A Robust Scalable Website Fingerprinting Technique. In *Proceedings of the 25th USENIX Security Symposium*. 1187–1203.
- [36] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 770–778.
- [37] Sébastien Henri, Ginés García, Pablo Serrano, Albert Banchs, and Patrick Thiran. 2020. Protecting against website fingerprinting with multihoming. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2020, issue 2. Sciend, 89–110.
- [38] Jordan Holland, Paul Schmitt, Nick Feamster, and Prateek Mittal. 2021. New directions in automated traffic analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3366–3383.
- [39] James K Holland, Jason Carpenter, Se Eun Oh, and Nicholas Hopper. 2023. DeTorrent: An Adversarial Padding-only Traffic Analysis Defense. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2024, issue 1. 98–115.
- [40] James K Holland and Nicholas Hopper. 2022. RegulaTor: A straightforward website fingerprinting defense. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2022, issue 2. Sciend, 344–362.
- [41] Rob Jansen. 2026. CellShift: RTT-Aware Trace Transduction for Real-World Website Fingerprinting. In *Proceedings of the 33rd Network and Distributed System Security Symposium*.
- [42] Rob Jansen and Ryan Wails. 2023. Data-explainable website fingerprinting with network simulation. *Proceedings on Privacy Enhancing Technologies* (2023).
- [43] Rob Jansen, Ryan Wails, and Aaron Johnson. 2026. A Measurement of Genuine Tor Traces for Realistic Website Fingerprinting. In *Passive and Active Measurement Conference*. Dataset available at <https://doi.org/10.5281/zenodo.10869889>.
- [44] Haifeng Jin, François Chollet, Qingquan Song, and Xia Hu. 2023. AutoKeras: An AutoML Library for Deep Learning. *Journal of Machine Learning Research* Vol. 24, 6 (2023), 1–6.
- [45] Zhaoxin Jin, Tianbo Lu, Shuang Luo, and Jiaye Shang. 2023. Transformer-based Model for Multi-tab Website Fingerprinting Attack. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1050–1064.
- [46] Marc Juárez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. 2014. A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 263–274.
- [47] Marc Juárez, Mohsen Imani, Mike Perry, Claudia Diaz, and Matthew Wright. 2016. Toward an efficient website fingerprinting defense. In *European Symposium on Research in Computer Security*. Springer, 27–46.
- [48] Aaron Klein, Stefan Falkner, Simon Bartels, Philipp Hennig, and Frank Hutter. 2017. Fast bayesian optimization of machine learning hyperparameters on large datasets. In *Proceedings of the Conference on Artificial Intelligence and Statistics*. 528–536.
- [49] Shuai Li, Huajun Guo, and Nicholas Hopper. 2018. Measuring Information Leakage in Website Fingerprinting Attacks and Defenses. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*.

- 1977–1992.
- [50] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2019. DARTS: Differentiable Architecture Search. In *Proceedings of the International Conference on Learning Representations*.
 - [51] Xiapu Luo, Peng Zhou, Edmond WW Chan, Wenke Lee, Rocky KC Chang, Roberto Perdisci, et al. 2011. HTTPOS: Sealing Information Leaks with Browser-side Obfuscation of Encrypted Flows. In *Proceedings of the 18th Annual Network and Distributed System Security*, Vol. 11.
 - [52] Navid Malekghaini, Elham Akbari, Mohammad A Salahuddin, Noura Limam, Raouf Boutaba, Bertrand Mathieu, Stephanie Moteau, and Stéphanie Tuffin. 2023. AutoML4ETC: Automated neural architecture search for real-world encrypted traffic classification. *IEEE Transactions on Network and Service Management* 21, 3 (2023), 2715–2730.
 - [53] Navid Malekghaini, Elham Akbari, Mohammad A Salahuddin, Noura Limam, Raouf Boutaba, Bertrand Mathieu, Stephanie Moteau, and Stéphane Tuffin. 2023. Deep learning for encrypted traffic classification in the face of data drift: An empirical study. *Computer Networks* 225 (2023), 109648.
 - [54] Nate Mathews, James K. Holland, Nicholas Hopper, and Matthew Wright. 2024. Laserbeak: Evolving Website Fingerprinting Attacks With Attention and Multi-Channel Feature Representation. *IEEE Transactions on Information Forensics and Security* 19 (2024), 9285–9300.
 - [55] Nate Mathews, James K Holland, Se Eun Oh, Mohammad Saidur Rahman, Nicholas Hopper, and Matthew Wright. 2023. SoK: A critical evaluation of efficient website fingerprinting defenses. In *Proceedings of the 44th IEEE Symposium on Security and Privacy*. 969–986.
 - [56] Asya Mitseva and Andriy Panchenko. 2024. Stop, don't click here anymore: boosting website fingerprinting by considering sets of subpages. In *Proceedings of the 33rd USENIX Security Symposium*. 4139–4156.
 - [57] Steven J Murdoch and George Danezis. 2005. Low-cost traffic analysis of Tor. In *Proceedings of the 26th IEEE Symposium on Security and Privacy*. 183–195.
 - [58] Milad Nasr, Alireza Bahramali, and Amir Houmansadr. 2021. Defeating DNN-Based Traffic Analysis Systems in Real-Time With Blind Adversarial Perturbations. In *Proceedings of the 30th USENIX Security Symposium*. 2705–2722.
 - [59] Rishab Nithyanand, Xiang Cai, and Rob Johnson. 2014. Glove: A bespoke website fingerprinting defense. In *Proceedings of the 13th ACM Workshop on Privacy in the Electronic Society*. 131–134.
 - [60] Se Eun Oh, Nate Mathews, Mohammad Saidur Rahman, Matthew Wright, and Nicholas Hopper. 2021. GANDaLF: GAN for Data-Limited Fingerprinting. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2021. 305–322.
 - [61] Se Eun Oh, Saikrishna Sunkam, and Nicholas Hopper. 2019. p1-FP: Extraction, Classification, and Prediction of Website Fingerprints with Deep Learning. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2019. 191–209.
 - [62] Andriy Panchenko, Fabian Lanze, Jan Pennekamp, Thomas Engel, Andreas Zinnen, Martin Henze, and Klaus Wehrle. 2016. Website Fingerprinting at Internet Scale. In *Proceedings of the 23rd Network and Distributed System Security Symposium*.
 - [63] Andriy Panchenko, Lukas Niessen, Andreas Zinnen, and Thomas Engel. 2011. Website fingerprinting in onion routing based anonymization networks. In *Proceedings of the 10th ACM Workshop on Privacy in the Electronic Society*. 103–114.
 - [64] Mike Perry. 2011. Experimental Defense Against Website Traffic Fingerprinting. <https://blog.torproject.org/experimental-defense-website-traffic-fingerprinting/>. Accessed: November 11, 2025.
 - [65] Mike Perry. 2013. A Critique of Website Traffic Fingerprinting Attacks. <https://blog.torproject.org/critique-website-traffic-fingerprinting-attacks/>. Accessed: November 11, 2025.
 - [66] Hieu Pham, Melody Guan, Barret Zoph, Quoc Le, and Jeff Dean. 2018. Efficient neural architecture search via parameters sharing. In *Proceedings of the International Conference on Machine Learning*. 4095–4104.
 - [67] Tobias Pulls and Rasmus Dahlberg. 2020. Website Fingerprinting with Website Oracles. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2020. 235–255.
 - [68] Mohammad Saidur Rahman, Mohsen Imani, Nate Mathews, and Matthew Wright. 2020. Mockingbird: Defending against deep-learning-based website fingerprinting attacks with adversarial traces. *IEEE Transactions on Information Forensics and Security* 16 (2020), 1594–1609.
 - [69] Mohammad Saidur Rahman, Payap Sirinam, Nate Mathews, Kantha Girish Gangadhara, and Matthew Wright. 2020. Tik-Tok: The utility of packet timing in website fingerprinting attacks. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2020. 5–24. Issue 3.
 - [70] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. 2019. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on artificial intelligence*, Vol. 33. 4780–4789.
 - [71] Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Sue-matsu, Jie Tan, Quoc V Le, and Alexey Kurakin. 2017. Large-scale evolution of image classifiers. In *Proceedings of the International Conference on Machine Learning*. 2902–2911.
 - [72] Vera Rimmer, Davy Preuveneers, Marc Juarez, Tom Van Goethem, and Wouter Joosen. 2018. Automated website fingerprinting through deep learning. In *Proceedings of the 25th Network and Distributed Systems Security Symposium*. Dataset available at <https://github.com/DistriNet/DLWF>.
 - [73] Shreyas Saxena and Jakob Verbeek. 2016. Convolutional neural fabrics. *Advances in Neural Information Processing Systems* 29 (2016).
 - [74] Shawn Shan, Arjun Nitin Bhagoji, Haitao Zheng, and Ben Y Zhao. 2021. Patch-based defenses against web fingerprinting attacks. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*. 97–109.
 - [75] Meng Shen, Kexin Ji, Zhenbo Gao, Qi Li, Liehuang Zhu, and Ke Xu. 2023. Subverting Website Fingerprinting Defenses with Robust Traffic Representation. In *Proceedings of the 32nd USENIX Security Symposium*. 607–624.
 - [76] Meng Shen, Jinhe Wu, Junyu Ai, Qi Li, Chenchen Ren, Ke Xu, and Liehuang Zhu. 2025. Swallow: A Transfer-Robust Website Fingerprinting Attack via Consistent Feature Learning. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 1574–1588.
 - [77] Vitaly Shmatikov and Ming-Hsiu Wang. 2006. Timing analysis in low-latency mix networks: Attacks and defenses. In *Proceedings of the 11th European Symposium on Research in Computer Security*. Springer, 18–33.
 - [78] Gurjot Singh, Rob Jansen, and Diogo Barradas. [n. d.]. Cricket's source code. <https://github.com/GurjotSingh02/cricket-1d-darts>.
 - [79] Prabhjot Singh, Shreya Arun Naik, Navid Malekghaini, Diogo Barradas, and Noura Limam. 2023. A First Look at Generating Website Fingerprinting Attacks via Neural Architecture Search. In *Proceedings of the 22nd ACM Workshop on Privacy in the Electronic Society*. 173–178.
 - [80] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. 2018. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1928–1943.
 - [81] Payap Sirinam, Nate Mathews, Mohammad Saidur Rahman, and Matthew Wright. 2019. Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1131–1148.
 - [82] Jean-Pierre Smith, Prateek Mittal, and Adrian Perrig. 2021. Website Fingerprinting in the Age of QUIC. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2021. 48–69.
 - [83] Alexander Veicht, Cedric Renggli, and Diogo Barradas. 2023. DeepSE-WF: Unified Security Estimation for Website Fingerprinting Defenses. In *Proceedings on Privacy Enhancing Technologies*, Vol. 2. 188–205.
 - [84] Linan Wang, Yiyang Zhao, Yu Jinai, Yuandong Tian, and Rodrigo Fonseca. 2019. AlphaX: eXploring neural architectures with deep neural networks and monte carlo tree search. *arXiv preprint arXiv:1903.11059* (2019).
 - [85] Tao Wang. 2020. High Precision Open-World Website Fingerprinting. In *Proceedings of the 41st IEEE Symposium on Security and Privacy*. 152–167.
 - [86] Tao Wang. 2021. The One-Page Setting: A Higher Standard for Evaluating Website Fingerprinting Defenses. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2794–2806.
 - [87] Tao Wang, Xiang Cai, Rishab Nithyanand, Rob Johnson, and Ian Goldberg. 2014. Effective attacks and provable defenses for website fingerprinting. In *Proceedings of the 23rd USENIX Security Symposium*. 143–157.
 - [88] Tao Wang and Ian Goldberg. 2013. Improved website fingerprinting on Tor. In *Proceedings of the 12th ACM Workshop on Privacy in the Electronic Society*. 201–212.
 - [89] Tao Wang and Ian Goldberg. 2017. Walkie-Talkie: An efficient defense against passive website fingerprinting attacks. In *Proceedings of the 26th USENIX Security Symposium*. 1375–1390.
 - [90] Yanbin Wang, Haitao Xu, Zhenhao Guo, Zhan Qin, and Kui Ren. 2022. snWF: Website Fingerprinting Attack by Ensembling the Snapshot of Deep Learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 1214–1226.
 - [91] Tao Wei, Changhu Wang, Yong Rui, and Chang Wen Chen. 2016. Network morphism. In *Proceedings of the International Conference on Machine Learning*. 564–572.
 - [92] Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadepta Dey, and Frank Hutter. 2023. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727* (2023).
 - [93] Martin Wistuba, Ambrish Rawat, and Tejaswini Pedapati. 2019. A survey on neural architecture search. *Journal of Machine Learning Research* (2019).
 - [94] Sirui Xie, Hehui Zheng, Chunxiao Liu, and Liang Lin. 2019. SNAS: stochastic neural architecture search. *Proceedings of the International Conference on Learning Representations* (2019).
 - [95] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong. 2019. PC-DARTS: Partial Channel Connections for Memory-Efficient Architecture Search. In *ICCV*.
 - [96] Chris Ying, Aaron Klein, Eric Christiansen, Esteban Real, Kevin Murphy, and Frank Hutter. 2019. NAS-Bench-101: Towards reproducible neural architecture search. In *Proceedings of the International Conference on Machine Learning*. 7105–7114.
 - [97] Arber Zela, Thomas Elsken, Tonmoy Saikia, Yassine Marrakchi, Thomas Brox, and Frank Hutter. 2019. Understanding and robustifying differentiable architecture search. *arXiv preprint arXiv:1909.09656* (2019).

- [98] Arber Zela, Aaron Klein, Stefan Falkner, and Frank Hutter. 2018. Towards Automated Deep Learning: Efficient Joint Neural Architecture and Hyperparameter Search. In *Proceedings of the International Conference on Machine Learning AutoML Workshop*.
- [99] Ce Zhang, Wangyong Lv, Gang Liu, and Yufan Wang. 2025. Multidimensional spatiotemporal autocorrelation analysis theory based on Multi-observation spatiotemporal Moran's I and its application in resource allocation. *Earth Science Informatics* 18, 1 (2025), 36.
- [100] Heng Zhang, Ziqian Chen, Wei Xia, Gang Xiong, Gaopeng Gou, Zhen Li, Guangyan Huang, and Yunpeng Li. 2025. ANASETC: Automatic Neural Architecture Search for Encrypted Traffic Classification. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [101] Xiyuan Zhao, Xinhao Deng, Qi Li, Yunpeng Liu, Zhuotao Liu, Kun Sun, and Ke Xu. 2024. Towards Fine-Grained Webpage Fingerprinting at Scale. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 423–436.
- [102] Barret Zoph and Quoc V Le. 2016. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578* (2016).
- [103] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. 2018. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 8697–8710.

A CRICKET Model Architecture

The model architecture defined for CRICKET in the course of our experiments (see §4.2) is represented in Listing 1. The model is comprised by a fixed *stem* composed of a convolution, batch normalization, and ReLU layer that projects the input signal to an initial number of channels without altering its length. The stem's output is then processed by a sequence of four DARTS cells: a normal cell at the base resolution, two reduction cells that gradually decrease the temporal length while expanding the feature dimension, and a final normal cell that operates on the shortest representation. Each cell takes as input the outputs of the two preceding cells (s_0, s_1), which are first aligned through 1×1 convolutions or factorized reductions when their resolutions differ. After the final cell, global average pooling removes the temporal dimension, and a fully connected layer maps the resulting features to class probabilities.

Normal cells. These operate at a fixed temporal resolution and use a mix of dilated, separable, and standard convolutions to extract local burst-level features. Dilated convolutions (kernel-5, dilation-4) capture short gaps between Tor cells without reducing resolution, while separable and wider convolutions (kernels-3,-5,-7,-9) focus on burst boundaries and directional transitions within short sequences. The four intermediate nodes combine these filters at different scales and their outputs are concatenated to preserve timing precision and channel diversity across layers.

Reduction cells. Reduction cells apply stride-2 operations on selected edges to halve the temporal length and double the number of feature channels. This compression integrates information from consecutive Tor bursts while maintaining their relative order, helping the network recognize repeating upload–download alternations and shifts in burst timing. Inputs from the two preceding cells are aligned through 1×1 convolutions or a factorized reduction when the resolutions differ. The resulting output encodes a denser,

higher-level view of traffic structure that supports later layers in distinguishing between website- or activity-specific burst patterns.

B Ethical Considerations on Dataset Usage

Our study involves the use of Tor traces from existing WF datasets; we did not conduct any new measurement for this paper. AWF is a synthetic dataset of the authors' traffic and is publicly available for download on GitHub [72]. GTT23 is a dataset of real Tor traces available on Zenodo [43]. The GTT23 measurement process and release plan was reviewed and approved by the Tor Research Safety Board. We followed the dataset's terms-of-use, keeping it private and using it for research only.

C Source Code and Data Availability

Source code. Our code for CRICKET is released as open-source [78]. Because CRICKET depends heavily on DARTS, our deliverable consists of the adaptation of DARTS to operate in the 1D setting, and wrappers to facilitate the ingestion of Tor traces.

Datasets and pre/post-processing code. The main datasets supporting our experiments (AWF, GTT23) have been released by their authors and are available online [43, 72]. Our open-source release includes ancillary code used to downsample and pre-process those datasets into the GTT_{100} , GTT_{200} , $OpenGTT_{200}$, and AWF_{100} subsets. We also released the post-processing code used to feed these data subsets into the WF defense simulators used in our work.

D Disclosure of AI Usage

We used generative AI-based tools to assist in the preparation of this paper. In particular, we used the tools to generate text for an initial description of the DARTS approach and NAS process for the background section (§2.3), to assist with formatting the equations presented in §2 and §3, and to assist with formatting the layout of the tables and setting figures' properties. We also used the tools to facilitate the integration of our described 1D primitives in the original DARTS source-code and workflow. We have manually verified and are responsible for the accuracy, originality, and integrity of the output of all AI-based tools.

Listing 1: End-to-end CRICKET architecture with cell/genotype details. Shapes are (channels, length) from our run.

```
Input (C_in, L0)
-> Stem: Conv+BN+ReLU -> (108, L0)
-> Cell 0 [Normal] pre: s0,s1 1x1Conv+BN -> C (108, L0) -> (144, L0)
-> Cell 1 [Reduction] pre: s0,s1 1x1Conv+BN -> 2C (144, L0) -> (288, L0/2)
-> Cell 2 [Reduction] pre: s0 FactRed(s2), s1 1x1Conv+BN -> 4C (288, L0/2) -> (576, L0/4)
-> Cell 3 [Normal] pre: s0 FactRed, s1 1x1Conv+BN -> 4C (576, L0/4) -> (576, L0/4)
-> Global Avg Pool (L0/4 -> 1) -> FC (channels -> #classes) -> Output probabilities

# genotype.normal (all stride 1)
N0 = dil_conv_5_d401 + sep_conv_300 N2 = conv_902 + conv_703
N1 = dil_conv_5_d401 + conv_502 N3 = conv_704 + conv_701
# genotype.reduce (sN = stride N)
N0 = sep_conv_300 (s2) + dil_conv_5_d401 (s2)
N1 = dil_conv_5_d402 (s1) + conv_500 (s2)
N2 = sep_conv_302 (s1) + conv_903 (s1)
N3 = conv_904 (s1) + dil_conv_5_d403 (s1)
# Every cell outputs Concat(N0..N3): 4x the preprocessed channels
```